

National SOC Platform

Production Readiness Assessment

Honest Technical Evaluation Report

Djezzy Telecom - Algeria

58%

Overall Readiness Score

NOT PRODUCTION READY

Document ID: SOC-PROD-READINESS-2026-001

Classification: CONFIDENTIAL - Internal Use Only

Assessment Date: August 22, 2026

Version: 1.0.0

Status: DRAFT - Pending Review

Prepared For: Djezzy SOC Team / ANRT Compliance

1. Executive Summary

This document provides an honest, unbiased assessment of the National SOC Platform's readiness for production deployment within Djezzy Telecom's infrastructure. The evaluation was conducted by analyzing the complete codebase, configuration files, security implementations, infrastructure definitions, and operational procedures against industry standards and best practices for Security Operations Center platforms.

1.1 Overall Assessment Verdict

VERDICT: NOT READY FOR PRODUCTION DEPLOYMENT

While the platform demonstrates solid architectural foundations and comprehensive feature coverage, several critical gaps must be addressed before production deployment. The current implementation represents approximately 58% of production readiness, with significant deficiencies in testing coverage, database configuration, environment hardening, and operational maturity that pose substantial risks to availability, security, and compliance posture.

1.2 Key Findings at a Glance

Category	Status	Score	Critical Issues
Security Posture	PARTIAL	65%	3 High, 5 Medium
Infrastructure	ADEQUATE	70%	2 High, 3 Medium
Testing & QA	CRITICAL GAP	15%	Zero test coverage
Database Layer	INCOMPLETE	45%	SQLite in prod config
Operations	DEVELOPING	55%	Missing runbooks
Compliance	PARTIAL	60%	ANRT gaps remain

1.3 Critical Blockers Summary

The following issues are classified as **CITICAL BLOCKERS** that must be resolved before any production deployment consideration. Each blocker represents a significant risk to system integrity, data security, or regulatory compliance:

BLK-001: Zero Test Coverage No unit tests, integration tests, or E2E tests exist in the codebase. This represents a catastrophic risk for production stability. Severity: CRITICAL
BLK-002: Production Database Misconfiguration

.env.example shows SQLite as default database. PostgreSQL schema exists but is not configured as primary.

Severity: CRITICAL

BLK-003: Placeholder Secrets in Configuration

JWT_SECRET, ANONYMIZATION_SALT, ENCRYPTION_KEY all contain placeholder values that would pass CI but fail in production.

Severity: CRITICAL

BLK-004: Missing Edge Middleware

No Next.js middleware.ts exists for route-level protection, allowing unauthenticated access to protected routes.

Severity: HIGH

BLK-005: Dockerfile Reference Mismatch

docker-compose.prod.yml references Dockerfile.production but actual file is named Dockerfile.

Severity: HIGH

2. Security Posture Assessment

The security assessment evaluates authentication mechanisms, authorization controls, data protection, input validation, and compliance with OWASP Top 10 and ANSSI security recommendations. The platform demonstrates strong architectural security patterns but has implementation gaps that require attention.

2.1 Authentication & Authorization (Score: 75/100)

Strengths Identified:

The platform implements a robust JWT-based authentication system with role-based access control (RBAC). The `api-auth.ts` middleware provides proper token verification via Authorization headers or HTTP-only secure cookies, rejecting insecure query-parameter tokens. Permission-based access control is implemented through a fine-grained permissions system supporting both individual permissions and role-level access. The authentication wrapper (`withAuth`) properly integrates with API handlers and returns standardized error responses with appropriate HTTP status codes (401 for unauthorized, 403 for forbidden).

Gaps & Concerns:

1. No `Next.js middleware.ts` exists for edge-level route protection
2. Session management lacks centralized invalidation mechanism
3. Token refresh flow implementation incomplete
4. Missing account lockout after failed authentication attempts
5. LDAP/SAML integration exists but lacks production testing validation

2.2 Rate Limiting Implementation (Score: 80/100)

The unified rate limiting system is well-architected with multiple algorithms including sliding window for standard endpoints, token bucket for streaming/SSE connections, and fixed window for export operations. The higher-order function (HOF) pattern allows clean integration with existing API handlers. Category-specific presets provide appropriate limits: auth endpoints (5/15min), data endpoints (100/min), export (3/hour), and stream connections (5 concurrent). Redis-backed storage supports multi-instance deployments with graceful in-memory fallback when Redis is unavailable.

Endpoint Type	Limit	Window	Algorithm
Authentication	5 requests	15 minutes	Sliding Window
Data APIs	100 requests	1 minute	Sliding Window
Data Export	3 requests	1 hour	Fixed Window
Streaming/SSE	5 concurrent	60s check	Token Bucket
Admin APIs	50 requests	1 minute	Sliding Window
Telecom/SS7	80 requests	1 minute	Sliding Window
Analytics/ML	50 requests	1 minute	Sliding Window

2.3 Data Protection & PII Handling (Score: 70/100)

The PII anonymization module demonstrates strong GDPR/ANSSI compliance awareness with SHA-256 salted hashing. The implementation includes specialized functions for IP addresses (preserving prefix for geo-analysis), emails (preserving domain), phone numbers (preserving country/area codes), and names (full hashing). Production safeguards validate salt strength (minimum 32 characters) and reject placeholder values. However, integration of anonymization into actual API response pipelines appears incomplete, meaning PII may still be exposed in raw form despite the module's existence.

2.4 Security Headers (Score: 85/100)

The Next.js configuration implements comprehensive OWASP-recommended security headers including X-Frame-Options (DENY), X-Content-Type-Options (nosniff), strict Referrer-Policy, Permissions-Policy disabling camera/microphone/ geolocation, and Content-Security-Policy with appropriate restrictions. HSTS and Expect-CT headers are conditionally applied in production. API responses explicitly disable caching to prevent sensitive data leakage through CDN or proxy caches. This represents a strong implementation meeting most security header best practices.

2.5 Security Vulnerabilities Identified

ID	Vulnerability	Severity	Status
SEC-001	Missing Input Validation on Some Endpoints	HIGH	Open
SEC-002	CSP Allows unsafe-inline in Development	MEDIUM	Known
SEC-003	Error Messages May Leak Stack Traces	MEDIUM	Partially Fixed
SEC-004	No Request Size Limits on Upload Endpoints	HIGH	Open
SEC-005	Redis Connection Without TLS in Default Config	MEDIUM	Config Issue
SEC-006	Rate Limit Bypass Header Exists	LOW	By Design
SEC-007	Missing CORS Configuration in Code	MEDIUM	Env Dependent

3. Infrastructure & DevOps Assessment

This section evaluates containerization, orchestration, CI/CD pipelines, monitoring capabilities, and deployment procedures. The platform shows mature infrastructure-as-code practices but has critical configuration mismatches that would prevent successful deployment.

3.1 Containerization (Score: 65/100)

Dockerfile Strengths:

The multi-stage Dockerfile follows best practices with: Alpine-based images for minimal attack surface, non-root user execution (nextjs user with UID 1001), proper health checks for Kubernetes liveness/readiness probes, timezone configuration for Algeria (Africa/Algiers), Prisma client generation at build time, and standalone output mode for optimized production builds. The build process correctly separates dependencies, compilation, and runtime stages.

Critical Issues:

- 1. **DOCKERFILE MISMATCH:** docker-compose.prod.yml references "Dockerfile: Dockerfile.production" but the actual file is named "Dockerfile". This will cause immediate build failure.
- 2. No .dockerignore file detected, potentially bloating build context
- 3. Missing security scanning stage in build pipeline
- 4. No image signing or vulnerability baseline established

3.2 Orchestration & Kubernetes (Score: 72/100)

Kubernetes manifests demonstrate production-awareness with proper namespace isolation (soc-backend), PodDisruptionBudgets for availability guarantees, HorizontalPodAutoscaling for traffic scaling, network policies implementing zero-trust principles, RBAC configurations following least-privilege, PodSecurity Standards enforcement, and resource quotas preventing noisy-neighbor scenarios. Helm charts support multiple environments (staging, production) with appropriate value overrides.

Component	Status	Notes
Deployments	Complete	Multi-container with affinity rules
Services	Complete	ClusterIP + Ingress configured
ConfigMaps	Complete	Environment-specific configs
Secrets	Template Only	Requires external secret management
Ingress	Complete	TLS termination configured
HPA	Complete	CPU/Memory based scaling (2-10 replicas)
Network Policies	Complete	Default-deny with explicit allow
RBAC	Complete	Service accounts + role bindings
PDB	Complete	Min available: 2 during disruptions

3.3 CI/CD Pipeline (Score: 68/100)

The GitLab CI/CD pipeline demonstrates enterprise-grade structure with nine distinct stages: validate, test, security, build, deploy-staging, deploy-production, verify, and notify. Security scanning includes Trivy for container vulnerabilities, TruffleHog for secrets detection, and kube-linter for Kubernetes configuration validation. Deployment uses Helm charts with database backup pre-deploy hooks and automatic rollback on health check failure. Slack notifications provide pipeline status updates, and ANRT compliance logging captures deployment events for audit trails.

Pipeline Gaps:

1. Unit test stage references non-existent test files (zero coverage)
2. No performance/regression testing stage
3. Security scans set to allow_failure: true (don't block deployment)
4. No chaos engineering or failure injection testing
5. Missing blue-green or canary deployment strategy
6. No automated rollback triggers based on error rates

3.4 Monitoring & Observability (Score: 62/100)

Monitoring infrastructure includes Prometheus for metrics collection with SOC-specific alerting rules, Grafana dashboards for platform overview visualization, and health check endpoints (/api/health) for probe targets. The Redis client includes built-in health checking with server info retrieval and response time metrics. Application logging uses structured formats with log levels appropriate for production debugging.

Observability Gaps:

1. No distributed tracing implementation (Jaeger/Zipkin)
2. Missing error tracking service integration (Sentry/Rollbar)
3. No synthetic monitoring or uptime checks configured
4. Alert runbooks not documented for on-call responders
5. Log aggregation strategy unclear (ELK stack defined but not integrated)
6. No business metrics dashboards for executive reporting

4. Database Layer Assessment

The database layer evaluation examines schema design, ORM usage, migration strategies, and production readiness of data persistence. This area reveals one of the most critical gaps requiring immediate remediation.

4.1 Schema Design (Score: 78/100)

The Prisma schema demonstrates comprehensive domain modeling with 42 models covering six domains: Authentication & Authorization, Core SOC Operations, Threat Intelligence, Telecommunications/SS7, Compliance (ARTP/ANSSI), and Analytics/ML. The schema implements proper relationships with referential integrity, enum types for constrained values, indexes for query optimization, and timestamps for audit trails. The PostgreSQL-optimized variant includes UUID primary keys, JSONB fields for flexible data, GIN indexes for JSON queries, and composite indexes for common query patterns.

4.2 CRITICAL: Database Provider Configuration (Score: 25/100)

BLOCKING ISSUE DETECTED

The current configuration specifies SQLite as the database provider:

```
datasource db { provider = "sqlite" url = env("DATABASE_URL") }
```

And the .env.example defaults to:

```
DATABASE_URL=file:./db/custom.db
```

This is UNACCEPTABLE for production deployment because:

1. SQLite does not support concurrent writes (locks entire database file)
2. No network access - cannot scale across multiple pods/instances
3. No built-in replication or failover capabilities
4. Performance degradation with databases > 1GB
5. Does not support the JSONB indexes defined in the PostgreSQL schema variant
6. Violates ANRT requirements for enterprise telecom platforms

A complete PostgreSQL schema (schema-postgresql.prisma) EXISTS but is NOT CONFIGURED as the active schema. Migration scripts and conversion tools also exist but have not been executed.

4.3 Migration Strategy Assessment

The project includes comprehensive migration tooling: SQL generation script (~900 lines) for converting SQLite schema to PostgreSQL, 5-phase migration guide documentation, Prisma migration runner with status/validation/health commands, and enterprise partitioning setup scripts for time-series data. However, none of this tooling has been validated against a real PostgreSQL instance, and the migration path from development (SQLite) to production (PostgreSQL) remains theoretical rather than proven.

4.4 Data Seeding & Demo Data (Score: 85/100)

The seed data implementation is thorough with 5000+ records across all tables: 12 diverse users with realistic roles, 55+ alerts covering all severity levels with MITRE ATT&CK; mappings, 15 incidents with proper lifecycle states, 25 threat indicators from various sources, 30 SS7 messages representing real fraud scenarios, 18 compliance items for ARTP/ANSSI frameworks, and 5000+ health metrics for dashboard

population. The data accurately reflects Algerian telecom context (Djezzy, Algiers coordinates, +213 phone prefixes) making it valuable for demonstrations and testing.

5. Testing & Quality Assurance Assessment

5.1 TEST COVERAGE: CRITICAL FAILURE (Score: 5/100)

CRITICAL GAP IDENTIFIED

A comprehensive search of the codebase revealed **ZERO test files**. No unit tests, no integration tests, no end-to-end tests, and no API contract tests exist. The patterns searched include:

- *.test.ts / *.test.tsx / *.test.js
- *.spec.ts / *.spec.tsx / *.spec.js
- __tests__ directories
- jest.config.* / vitest.config.* / playwright.config.*

This represents a catastrophic gap in production readiness. Without tests, the following risks are unmitigated:

1. Regression bugs cannot be detected during development
2. Code review cannot validate correctness without executable specifications
3. Refactoring becomes extremely high-risk
4. CI/CD pipeline test stage will always fail or be skipped
5. No safety net for the complex security logic (auth, rate limiting, PII handling)
6. Database migrations cannot be tested before production application
7. API contract compatibility cannot be verified across versions

5.2 Required Test Coverage Matrix

Test Layer	Priority	Current	Target	Key Areas
Unit Tests	P0 - CRITICAL	0%	80%+	Auth, Rate Limit, Anonymization, Validation
Integration Tests	P0 - CRITICAL	0%	70%+	API routes, DB queries, Redis ops
E2E Tests	P1 - HIGH	0%	60%+	User flows, Alert lifecycle, Login
Contract Tests	P1 - HIGH	0%	100%	API schemas, Response formats
Performance Tests	P1 - HIGH	Scripts only	Baseline	Load, Stress, Soak testing
Security Tests	P0 - CRITICAL	0%	100%	Auth bypass, Injection, XSS, CSRF

5.3 Load Testing Artifacts (Score: 45/100)

Load testing scripts exist using k6 and JMeter frameworks covering: API stress testing, concurrent user simulation, ingestion throughput measurement, soak testing for memory leak detection, and dashboard load profiling. However, these scripts appear to be templates without execution history, baselines, or pass/fail criteria. No performance SLAs are defined (e.g., p95 latency < 200ms, max throughput > 1000 req/s), making it impossible to determine if the platform meets production performance requirements.

6. Operational Readiness Assessment

Operational readiness evaluates whether the team can effectively operate, maintain, and troubleshoot the platform in production. This encompasses documentation, runbooks, on-call procedures, backup strategies, and incident response capabilities.

6.1 Documentation Status (Score: 55/100)

Document	Status	Quality
Architecture Specification	Exists	Comprehensive
API Reference	Partial	Needs updating
Deployment Guide	Exists	Good
Operations Manual	Draft	Incomplete
Runbooks (7 total)	Exist	Template quality
Training Materials (4)	Exist	Basic
Security Report	Exists	Detailed
Compliance Checklist	Exists	ARTP-focused
Troubleshooting Guide	Missing	N/A
Capacity Planning Guide	Missing	N/A

6.2 Incident Response Readiness (Score: 50/100)

Incident response playbooks exist covering standard SOC scenarios with escalation matrices and shift handover procedures. The platform integrates with SOAR tools (TheHive/Cortex) for automated response orchestration. However, critical gaps remain: no defined RTO/RPO objectives, no disaster recovery plan tested, no incident post-mortem template, and no communication templates for stakeholder notifications during outages. The incident command center UI component exists but lacks integration with ticketing systems.

6.3 Backup & Disaster Recovery (Score: 35/100)

SIGNIFICANT GAPS IN BUSINESS CONTINUITY PLANNING

Current state analysis reveals:

1. No automated database backup configuration in Kubernetes manifests
2. No backup retention policy defined
3. No recovery time objective (RTO) established
4. No recovery point objective (RPO) established
5. No DR site or multi-region deployment strategy
6. No backup encryption at rest verification
7. No restore testing procedure or history

For a national SOC platform handling sensitive telecommunications security data, this level of DR preparedness is insufficient. A complete outage could result in loss of threat visibility during active attacks, with potential regulatory penalties from ANRT.

6.4 Environment & Configuration Management (Score: 40/100)

The .env.example file provides good documentation of required variables, but critical issues persist:

Issues Requiring Immediate Attention:

1. Placeholder values for ALL security-critical secrets (JWT_SECRET, REFRESH_SECRET, ENCRYPTION_KEY, CSRF_SECRET, ANONYMIZATION_SALT)
2. No .env.validation script to enforce required variables at startup
3. No secrets management integration (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets encryption)
4. Development defaults that could accidentally reach production
5. ALLOW_MFA_BYPASS flag exists (should never be in codebase)
6. No environment-specific configuration files (.env.staging, .env.production)

7. Regulatory Compliance Assessment

As Algeria's national telecommunications security platform, compliance with ARTP (Autorite de Regulation des Postes et des Telecommunications) and ANSSI (French cybersecurity agency, given historical ties) frameworks is mandatory. This section evaluates alignment with these requirements plus international standards like ISO 27001 and GDPR (for any EU subscriber data processed).

7.1 ARTP Compliance Status (Score: 60/100)

The platform includes dedicated ARTP framework implementation with compliance engine, scoring calculations, and reporting templates. Coverage includes: lawful interception logging interfaces, data localization (Algeria-only infrastructure), encryption standards compliance, and audit trail requirements. However, formal ARTP certification has not been obtained, and some requirements around data retention periods and government access protocols need clarification with legal counsel.

7.2 ANSSI Security Framework (Score: 55/100)

ANSSI framework integration exists with enhanced security controls covering: identity and access management hardening, cryptographic agility (algorithms can be swapped), security event correlation, and vulnerability management integration. The framework provides scoring mechanisms for continuous compliance monitoring. Gap: Third-party ANSSI audit has not been conducted, and formal certification pathway is undefined.

7.3 Compliance Requirements Matrix

Requirement	Source	Status	Gaps
Data Localization	ARTP Art. 12	Partial	Cloud provider verification needed
Encryption at Rest	ANSSI PDI	Implemented	Key rotation automation needed
Encryption in Transit	ANSSI PDI	Implemented	TLS 1.3 enforcement partial
Access Control	ISO 27001 A.9	Implemented	Privileged access review missing
Audit Logging	ARTP Art. 15	Implemented	Log integrity verification needed
Incident Reporting	ARTP Art. 18	Partial	Timeline to ARPT undefined
Data Retention	ARTP Art. 14	Not Implemented	Policy and automation needed
PII Protection	GDPR Art. 32	Partial	DPO appointment needed
Business Continuity	ISO 27001 A.17	Not Implemented	Full DR plan required
Vendor Risk Mgmt	ISO 27001 A.15	Partial	Third-party audit pending

8. Remediation Roadmap

The following roadmap provides a prioritized, phased approach to achieving production readiness. Items are sequenced by dependency order and risk reduction impact. Estimated effort assumes a team of 2-3 experienced full-stack developers with DevOps capabilities.

Phase 1: Critical Blockers (Weeks 1-2) - ESTIMATED: 40 hours

1.1	Fix Dockerfile reference mismatch in docker-compose.prod.yml	2h	BLOCKER
1.2	Configure PostgreSQL as primary database provider	8h	BLOCKER
1.3	Execute database migration to PostgreSQL	8h	BLOCKER
1.4	Generate production secrets (all placeholders)	2h	BLOCKER
1.5	Add Next.js middleware.ts for route protection	8h	HIGH
1.6	Create .env.validation startup script	4h	HIGH
1.7	Add request size limits to all upload endpoints	4h	HIGH
1.8	Implement input validation middleware	8h	CRITICAL

Phase 2: Testing Foundation (Weeks 3-4) - ESTIMATED: 60 hours

2.1	Set up Vitest/Jest testing framework	4h	CRITICAL
2.2	Write unit tests for auth module (target: 80%+)	12h	CRITICAL
2.3	Write unit tests for rate limiting	6h	HIGH
2.4	Write unit tests for PII anonymization	6h	HIGH
2.5	Set up Playwright for E2E testing	6h	HIGH
2.6	Write E2E tests for login/auth flows	8h	HIGH
2.7	Write integration tests for API endpoints	12h	HIGH
2.8	Configure CI to require passing tests	4h	CRITICAL
2.9	Add security test cases (injection, XSS)	8h	CRITICAL

Phase 3: Operations Hardening (Weeks 5-6) - ESTIMATED: 40 hours

3.1	Implement automated database backups (pg_dump cron)	6h	HIGH
3.2	Define and document RTO/RPO objectives	4h	HIGH
3.3	Write DR runbook and test restore procedure	8h	HIGH
3.4	Integrate Sentry/error tracking service	6h	MEDIUM
3.5	Configure structured log aggregation (ELK)	8h	MEDIUM

3.6	Document on-call procedures and escalation	4h	MEDIUM
3.7	Set up synthetic monitoring (uptime checks)	4h	MEDIUM

Phase 4: Compliance & Validation (Weeks 7-8) - ESTIMATED: 32 hours

4.1	Engage third-party penetration tester	0h	EXTERNAL
4.2	Address pen test findings	16h	VARIES
4.3	Complete ARTP compliance documentation	8h	HIGH
4.4	Execute load tests and establish baselines	8h	HIGH
4.5	Conduct disaster recovery drill	8h	HIGH
4.6	Final security review and sign-off	4h	HIGH
4.7	Prepare Go/No-Go decision package	4h	HIGH

TOTAL REMEDIATION EFFORT: ~172 hours (8 weeks with 2-3 developers)

Upon completion of all four phases, the estimated production readiness score would increase from **58%** to approximately **85-90%**, which meets the minimum threshold for production deployment consideration. Remaining gaps would represent acceptable residual risks with appropriate mitigations and monitoring in place.

9. Conclusion & Recommendations

The National SOC Platform represents a technically sound architecture with comprehensive feature coverage for telecommunications security operations. The codebase demonstrates strong security awareness, modern development practices, and thoughtful integration of industry-standard tools. However, the current state falls significantly short of production readiness due to critical gaps in testing, database configuration, and operational maturity.

9.1 Final Recommendation

RECOMMENDATION: DO NOT DEPLOY TO PRODUCTION

Based on this assessment, the National SOC Platform should **NOT** be deployed to production until the Phase 1 critical blockers are resolved and Phase 2 testing foundation is established. Deploying in the current state would expose Djezzy to unacceptable risks including: data loss from SQLite concurrency limitations, security breaches from placeholder credentials, inability to detect regressions, and potential ARTP compliance violations.

Recommended Path Forward:

1. Allocate 8 weeks for remediation with dedicated resources
2. Prioritize Phase 1 items as absolute prerequisites
3. Establish staging environment mirroring production
4. Execute full remediation roadmap before Go-Live decision
5. Engage third-party security auditor for validation
6. Conduct tabletop exercise with operations team
7. Perform gradual rollout with extensive monitoring

9.2 Acknowledgments

This assessment was conducted through comprehensive codebase analysis, configuration review, and architecture evaluation. The development team has built a solid foundation that, with focused remediation efforts, can become a production-ready national SOC platform serving Djezzy Telecom's security operations needs while meeting ARTP regulatory requirements.

Assessment Lead Date

August 22, 2026

Review Date

Pending Review